

Laborator 3 - CSS

CSS Selectors & Combinators

1 Selectorii de bază

- **Selector de tag (element)** – Selectează toate elementele de un anumit tip.

```
p {  
    color: blue;  
}
```

Afectează toate elementele `<p>` din pagină.

- **Selector de clasă (.)** – Selectează toate elementele care au o anumită clasă.

```
.table {  
    border: 1px solid black;  
}
```

Poți aplica mai multe clase unui element:

```
<div class="table bordered"></div>
```

- **Selector de ID (#)** – Selectează un singur element cu un ID specific.

```
#main-header {  
    background-color: gray;  
}
```

⚠ ID-urile trebuie să fie unice în pagină!

- **Selector universal (*)** – Se aplică tuturor elementelor din pagină.

```
* {  
    margin: 0;  
    padding: 0;  
}
```

2 Combinatori CSS

Combinatorii definesc relațiile dintre selectori.

Combinator	Descriere	Exemplu
Descendent ()	Selectează elementele din interiorul altuia	<code>div p { color: red; }</code> → Toate <code><p></code> din <code><div></code>
Copil direct (>)	Selectează doar copiii direcți ai unui element	<code>div > p { color: red; }</code>
Frate general (~)	Selectează toate elementele de același nivel care vin după un alt element	<code>h2 ~ p { color: green; }</code>
Frate imediat	Selectează doar primul element de același nivel	<code>h2 + p { font-weight: bold; }</code>

Combinator	Descriere	Exemplu
(+)	care vine imediat după altul	

⌘ Exemple practice

Descendent ()

```
div p {
  color: red;
}
```

```
<div>
  <p>Acest text este roșu.</p>
</div>
```

Copil direct (>)

```
div > p {
  color: blue;
}
```

```
<div>
  <p>Acest text va fi albastru.</p>
  <section>
    <p>Acest text NU va fi afectat.</p>
  </section>
</div>
```

Frate imediat (+)

```
h2 + p {
  color: green;
}
```

```
<h2>Titlu</h2>
<p>Acest paragraf devine verde.</p>
<p>Acest paragraf NU este afectat.</p>
```

Frate general (~)

```
h2 ~ p {
  font-style: italic;
}
```

```
<h2>Titlu</h2>
<p>Toate paragrafele după `h2` vor fi italice.</p>
<p>Inclusiv acesta!</p>
```

📌 Rezumat

- ✓ **Tag selector** (`p`) – Selectează toate elementele de un tip.
- ✓ **Clasă** (`.table`) – Poate fi aplicată mai multor elemente.
- ✓ **ID** (`#id`) – Trebuie să fie unic.
- ✓ **Universal** (`*`) – Aplică stiluri tuturor elementelor.
- ✓ **Combinatori** – Definesc relațiile între elemente.

👉 Folosește combinatorii pentru un CSS mai eficient și organizat!

🎨 Pseudo-elemente și selectori de atribute în CSS

◆ 1. Pseudo-elemente (`::before` , `::after` , etc.)

Pseudo-elementele permit stilizarea anumitor părți ale unui element, fără a adăuga elemente suplimentare în HTML.

📌 **Sintaxă:**

```
selector::pseudo-element {
  proprietate: valoare;
}
```

1 `::before` și `::after`

Acestea adaugă conținut înainte sau după un element. Necesită `content:` în CSS.

```
button::before {
  content: "🔥";
  margin-right: 5px;
}

button::after {
  content: "✅";
  color: green;
}
```

```
<button>Click me</button>
```

2 `::first-letter` și `::first-line`

- `::first-letter` – Stilizează prima literă a unui element.
- `::first-line` – Stilizează prima linie a unui element (depinde de dimensiunea containerului).

```
p::first-letter {  
    font-size: 2em;  
    color: red;  
}  
  
p::first-line {  
    font-weight: bold;  
}
```

`<p>`Acesta este un paragraf lung. Prima literă va fi mai mare, iar prima linie va fi bold.`</p>`

3 `::selection`

Stilizează textul selectat de utilizator.

```
::selection {  
    background: yellow;  
    color: black;  
}
```

👉 Când utilizatorul selectează textul, acesta va avea fundal galben și text negru.

♦ 2. Selectorii pentru atribute (`[atribut]`)

Acești selectori permit stilizarea elementelor în funcție de atributele lor.

📌 **Exemple:**

Selector	Descriere
<code>input[type="text"]</code>	Selectează toate <code><input></code> de tip text.
<code>a[target="_blank"]</code>	Selectează toate linkurile care se deschid într-o fereastră nouă.
<code>[disabled]</code>	Selectează toate elementele cu atributul <code>disabled</code> .
<code>input[placeholder]</code>	Selectează <code><input></code> care au un atribut <code>placeholder</code> .

✂ Exemple practice:

1 Stilizare pentru <input> de tip text

```
input[type="text"] {  
  border: 2px solid blue;  
  padding: 5px;  
}
```

```
<input type="text" placeholder="Introduceți textul aici">
```

2 Stilizare pentru linkuri externe (target="_blank")

```
a[target="_blank"] {  
  color: red;  
  text-decoration: underline;  
}
```

```
<a href="https://example.com" target="_blank">Link extern</a>
```

3 Stilizare pentru elementele dezactivate ([disabled])

```
button[disabled] {  
  background: gray;  
  color: white;  
  cursor: not-allowed;  
}
```

```
<button disabled>Nu poți da click</button>
```

4 Stilizare pentru placeholder într-un <input>

```
input::placeholder {  
  color: gray;  
  font-style: italic;  
}
```

```
<input type="text" placeholder="Scrie aici...">
```

📌 Rezumat

- ✅ **Pseudo-elemente:** `::before`, `::after`, `::first-letter`, `::first-line`, `::selection`
- ✅ **Selectori pentru attribute:** `[type="text"]`, `[disabled]`, `[placeholder]`, `[target="_blank"]`
- ✅ **Combinăția lor oferă un control mai avansat asupra stilizării!** 🎨 🔥

1 2 3 4 `calc()` în CSS

Funcția `calc()` este folosită pentru a efectua calcule matematice direct în CSS, combinând unități diferite (px, %, em, vw, etc.).

📌 Sintaxă:

```
proprietate: calc( expresie matematică );
```

🔗 Exemple practice

1 Combinarea unităților

```
.box {  
  width: calc(100% - 50px);  
  height: calc(50vh - 20px);  
}
```

📌 Explicație:

- Lățimea va fi 100% din container **minus 50px**.
- Înălțimea va fi **jumătate din viewport height (vh) - 20px**.

2 Operații matematice

`calc()` suportă:

- ✅ **Adunare (+)**
- ✅ **Scădere (-)**
- ✅ **Înmulțire (*)**
- ✅ **Împărțire (/)**

```
.container {  
  padding: calc(10px + 2em);  
  font-size: calc(1rem * 1.5);  
}
```

📌 Explicație:

- `padding` combină px și em.
- `font-size` este de 1.5 ori mai mare decât 1rem.

3 Poziționare flexibilă

```
.sidebar {  
  width: calc(50vw - 100px);  
  left: calc(100% - 300px);  
}
```

📌 Explicație:

- Sidebar-ul va avea jumătate din lățimea ecranului **minus 100px**.
- `left` îl poziționează exact la **100% din container - 300px**.

◆ Reguli importante pentru `calc()`

1. Spații obligatorii între operatori!

❌ `width: calc(100%-50px);` (Greșit ❌)

✅ `width: calc(100% - 50px);` (Corect ✅)

2. Poate fi folosit în orice proprietate numerică (width, height, padding, margin, font-size, etc.).

3. Funcționează bine cu variabile CSS (`var(--varName)`)

```
:root {  
  --spacing: 20px;  
}  
  
.element {  
  margin: calc(var(--spacing) * 2);  
}
```

📌 Rezumat

- ✅ `calc()` face calcule dinamice în CSS
- ✅ Poate combina **unități diferite** (`px` , `%` , `em` , `vw` , etc.)
- ✅ Se poate folosi la **width, height, padding, margin, font-size** etc.
- ✅ **Spațiile între operatori sunt obligatorii!**

💡 Folosește `calc()` pentru layout-uri responsive și flexibile! 🚀